



MEI/MCA/MIA/MMC

Computação Paralela

2025/26

João Luís Sobral

Course foundations



- **Lecturing/teaching team**

- Theoretical lectures: João L. Sobral
- Lab classes: J. Sobral, Rui Silva, Bernardo Malaca (labs/project support **only**)

- **Documentation & lectures/classes**

- Documentation (slides, lab guides, ...): English, only on **MEI** BB
- Project reports, usually English (but not mandatory)
- Oral presentations and defence: optional, but usually Portuguese

BB Key:
cp2526

- **Expected background**

- Imperative Programming and Computer Architecture
- Lab skills in Unix, C programming and debugging

- **Assessment**

- 1x written test/exam (test ??, weight: 40%, **MINIMUM GRADE 5.0**)
- 1x intermediate work w/ report (submission ??th Nov, weight: 10%)
- 1x final work w/ report & defence (??th Jan, weight: 50%)

Course foundations



• Lab classes and project support

- João Sobral (JS) – PL6 & PL5
- Rui Silva (RS) – PL4 & PL2
- Bernardo Malaca (BM) - Extra sessions & support
 - Some extra lab sessions (they will be announced)
 - Project presentation & bootstrapping
 - Project support (some scheduled & upon request)
 - Online & presential

Minimum 10
students per PL

• Resources

- Search cluster (required for PLs)
 - 2x **20-core Intel** & 2x GPUs **Tesla K20**
- Minho Advanced Computing Center
 - **48-core ARM** (A64FX processors)
 - NVidia **Ampere** GPUs (A100)

	segunda-feira	terça-feira	quarta-feira
8:00			
8:30			
9:00	Computação Paralela B - Edifício 1 - 0.08		
9:30			
10:00			
10:30			
11:00			
11:30			
12:00			
12:30			
13:00			
13:30			
14:00	Computação Paralela B - Edifício 2 - 2.05 PL6	Computação Paralela B - Edifício 2 - 0.14 PL4	
14:30			
15:00	JS	RS	
15:30			
16:00	Comp Paralela B - Edifício 1 - 0.08 PL5		Computação Paralela B - Edifício 7 - 1.10 PL2
16:30	JS	BM	RS
17:00			
17:30			
18:00			
18:30			
19:00			

Key textbook (1)

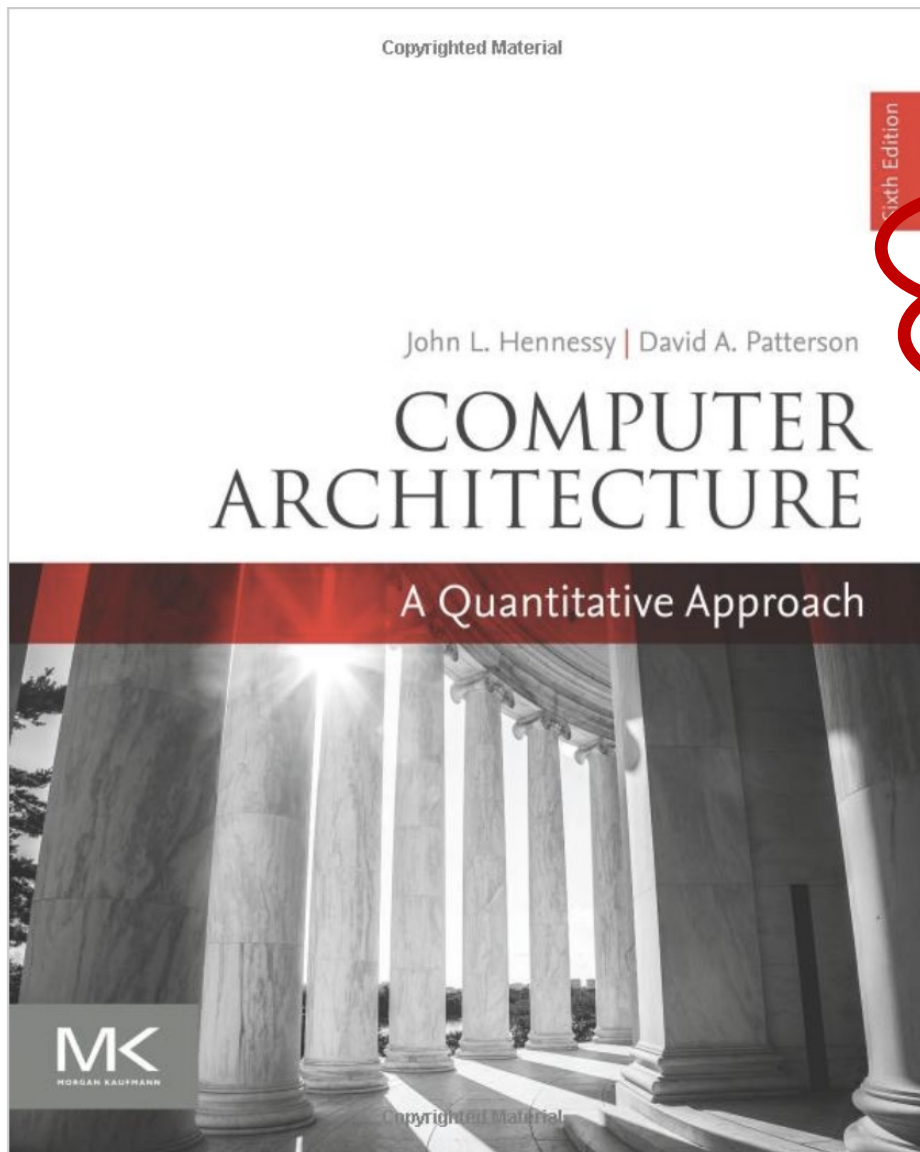


Table of Contents

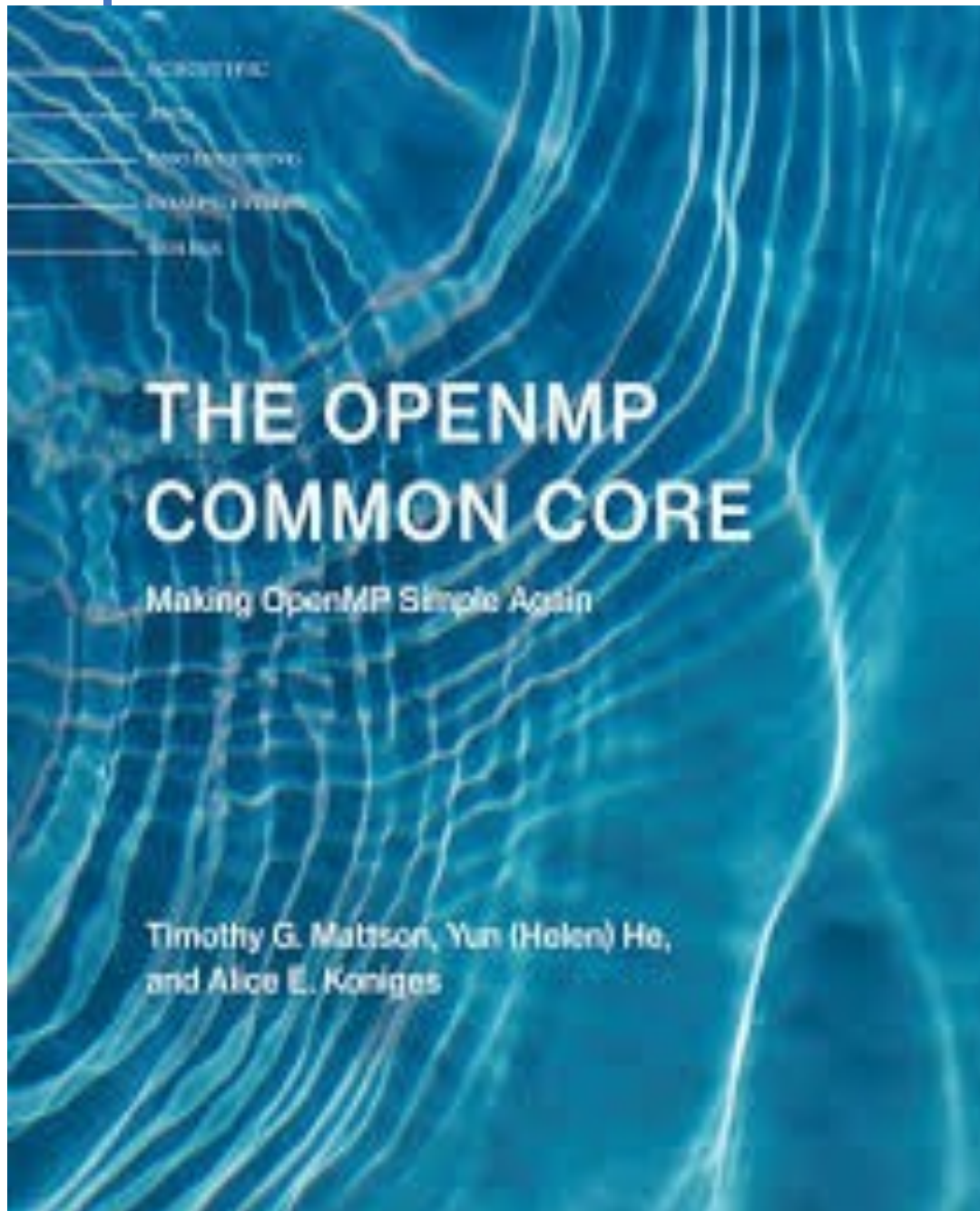
Printed Text

1. Fundamentals of Quantitative Design and Analysis
2. Memory Hierarchy Design
3. Instruction-Level Parallelism and Its Exploitation
4. Data-Level Parallelism in Vector, SIMD, and GPU Architectures
5. Multiprocessors and Thread-Level Parallelism
6. The Warehouse-Scale Computer
7. Domain-Specific Architectures
 - A. Instruction Set Principles
 - B. Review of Memory Hierarchy
 - C. Pipelining: Basic and Intermediate Concepts

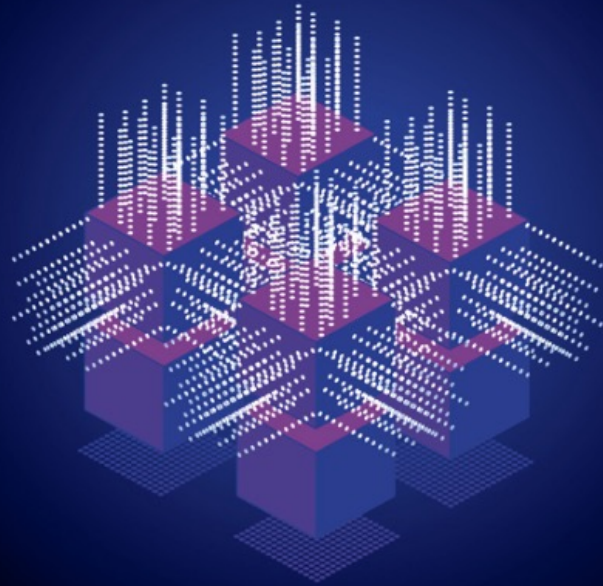
Online

- D. Storage Systems
- E. Embedded Systems
- F. Interconnection Networks
- G. Vector Processors
- H. Hardware and Software for VLIW and EPIC
- I. Large-Scale Multiprocessors and Scientific Applications
- J. Computer Arithmetic
- K. Survey of Instruction Set Architectures
- L. Advanced Concepts on Address Translation
- M. Historical Perspectives and References

Key textbook (2)



II	The OpenMP Common Core	
4	Threads and the OpenMP Programming Model	47
4.1	Overview of OpenMP	47
4.2	The Structure of OpenMP Programs	47
4.3	Threads and the Fork Join Pattern	50
4.4	Working with Threads	56
4.4.1	The SPMD Design Pattern	58
4.4.2	False Sharing	63
4.4.3	Synchronization	67
4.4.3.1	Critical	67
4.4.3.2	Barrier	69
4.5	Closing Comments	72
5	Parallel Loops	75
5.1	Worksharing-Loop Construct	76
5.2	Combined Parallel Worksharing-Loop Construct	79
5.3	Reductions	79
5.4	Loop Schedules	83
5.4.1	The Static Schedule	83
5.4.2	The Dynamic Schedule	84
5.4.3	Choosing a Schedule	86
5.5	Implicit Barriers and the Nowait Clause	90
5.6	Pi Program with Parallel Loop Worksharing	92
5.7	A Loop-Level Parallelism Strategy	94
5.8	Closing Comments	96
6	OpenMP Data Environment	99
6.1	Default Storage Attributes	100
6.2	Modifying Storage Attributes	102
6.2.1	The Shared Clause	103



PARALLEL COMPUTING ARCHITECTURES AND APIs

IoT Big Data Stream Processing

Vivek Kale



CRC Press
Taylor & Francis Group

A CHAPMAN & HALL BOOK

TABLE OF CONTENTS

1. Uniprocessor Computers
2. Processor Physics and Moore's Law
- Section I Genesis of Parallel Computing
3. Processor Basics
4. Networking Basics
5. Distributed Systems Basics
- Section II Road to Parallel Computing
6. Parallel Systems
7. Parallel Computing Models
8. Parallel Algorithms
- Section III Parallel Computing Architectures
9. Parallel Computing Architecture Basics
10. Shared Memory Architecture
11. Message-Passing Architecture
12. Stream Processing Architecture
- Section IV Parallel Computing Programming
13. Parallel Computing Programming Basics
14. Shared-memory Parallel Programming with OpenMP
15. Message Passing Parallel Programming with MPI
16. Stream Processing Programming with CUDA, OpenCL, and OpenACC
- Section V Internet of Things Big Data Stream Processing
17. Internet of Things (IoT) Technologies
18. Sensor Data Processing
19. Big Data Computing
20. Big Data Stream Processing
- Epilogue: Quantum Computing

Key textbook (4)

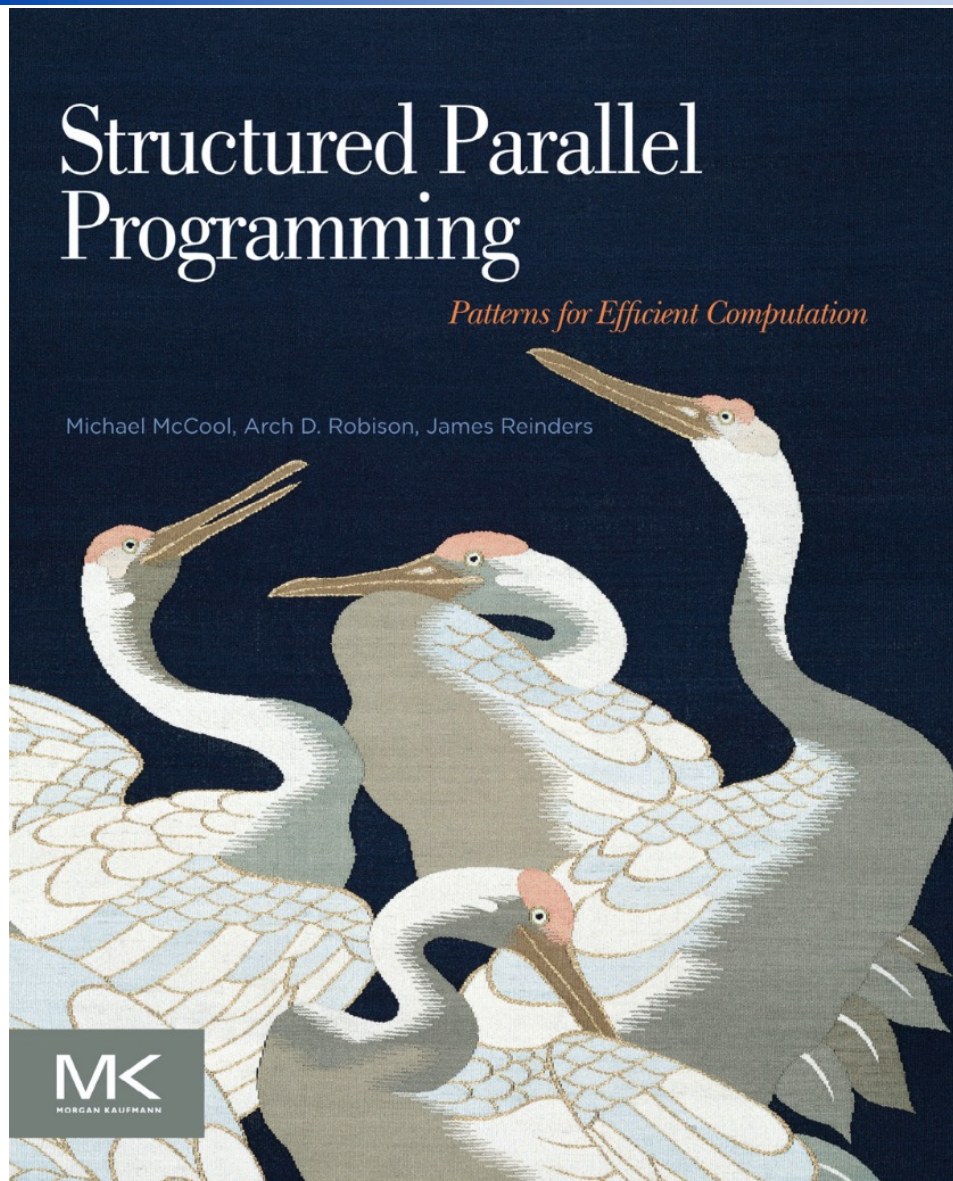
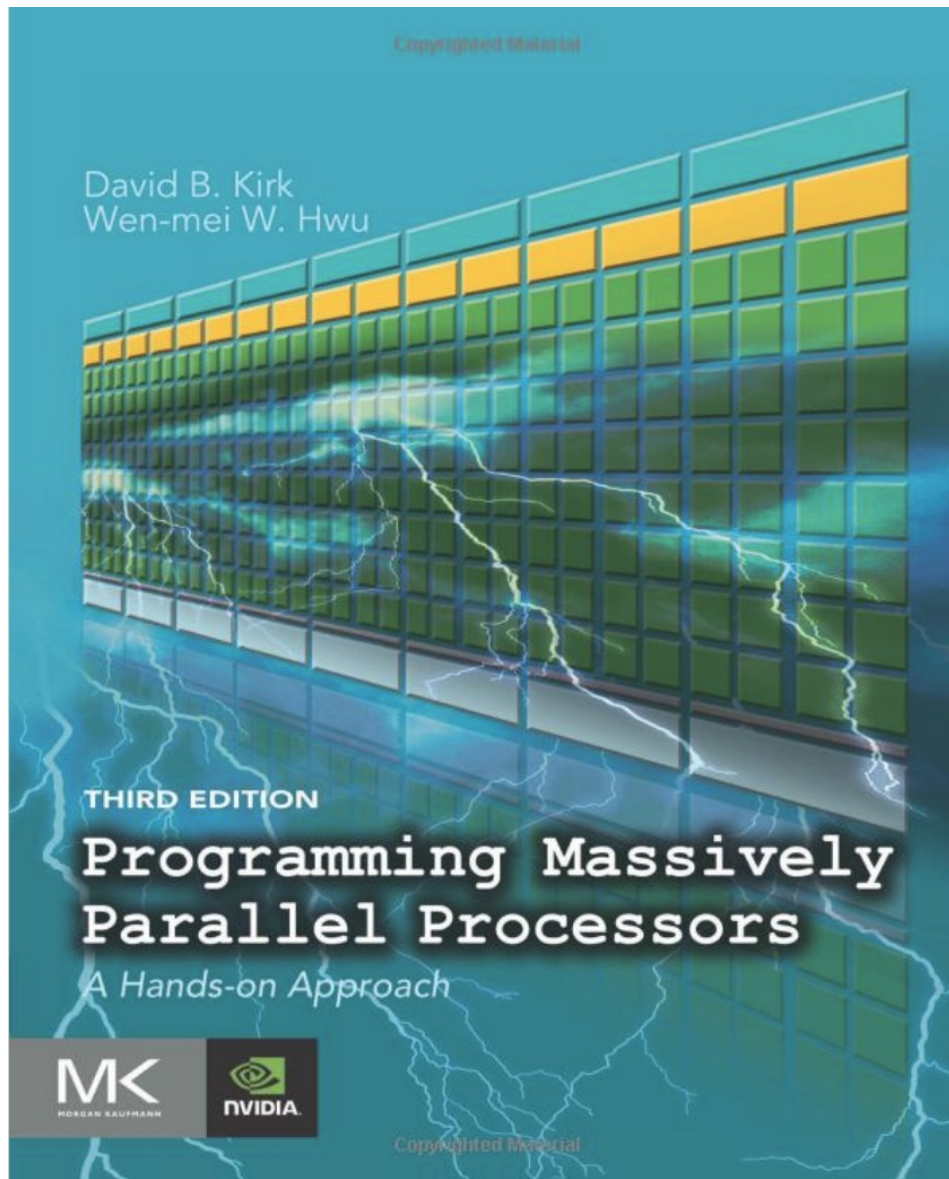


Table of contents

- 1 - Introduction
- 2 - Background
- 3 - Patterns
- 4 - Map
- 5 - Collectives
- 6 - Data Reorganization
- 7 - Stencil and Recurrence
- 8 - Fork-Join
- 9 - Pipeline
- 10 - Forward Seismic Simulation
- 11 - K-Means Clustering
- 12 - Bzip2 Data Compression
- 13 - Merge Sort
- 14 - Sample Sort
- 15 - Cholesky Factorization
- Appendices

Recommended textbook (1)



Contents

GPU & CUDA

1. Introduction
2. Data parallel computing
3. Scalable parallel execution
4. Memory and data locality
5. Performance considerations
6. Numerical considerations
7. Parallel patterns: Convolution
8. Parallel patterns: Prefix Sum
9. Parallel patterns : Parallel Histogram Computation
10. Parallel patterns: Sparse Matrix Computation
11. Parallel patterns: Merge Sort
12. Parallel patterns: Graph Searches
13. CUDA dynamic parallelism
14. Application case study—non-Cartesian magnetic ...
15. Application case study—molecular visualization ...
16. Application case study—machine learning
17. Parallel programming and computational thinking
18. Programming a heterogeneous computing cluster
19. Parallel programming with OpenACC
20. More on CUDA and graphics processing computing
21. Conclusion and outlook

Appendix A. An introduction to OpenCL

Appendix B. THRUST: a productivity-oriented library for
CUDA

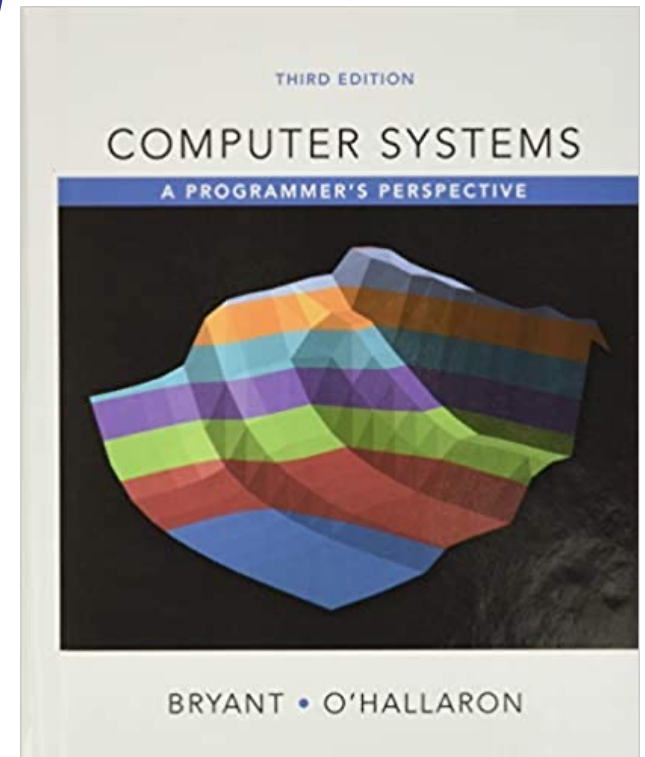
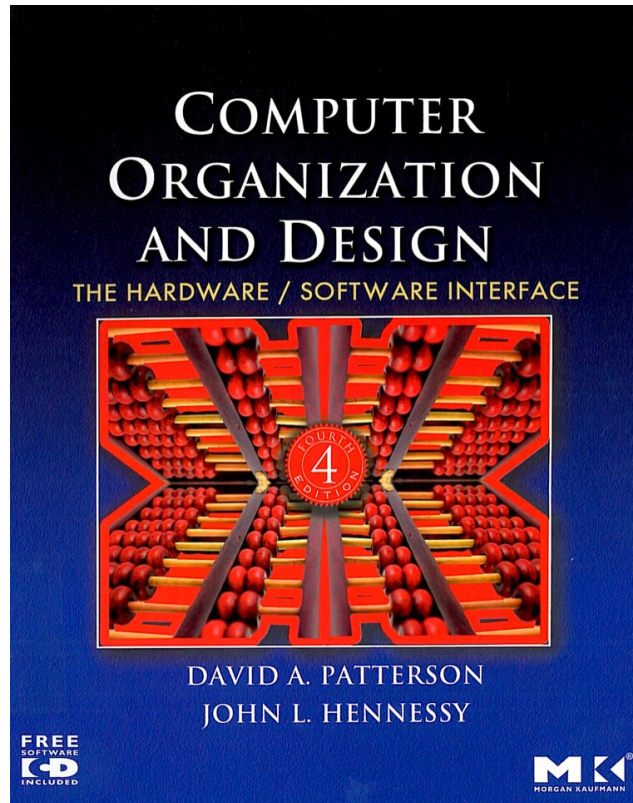
Background concepts from a basic Computer Systems course



Some notes/comments for this course:

– some slides are borrowed from

and some from



Focus on this course: performance engineering



Main goal

- Learn how to write programs that run **efficiently** on modern computers
- The only way to get there:
 - write programs that **do lots of independent things in parallel**

Why parallelism?

- Sequential performance stopped improving around 2000
- All new performance comes from **parallelism**
- New programs (code) are needed

Performance engineering

- **Work Assignment (60% of grade)**

"Solve this problem, on this computer, for this input, as fast as possible"

- write a parallel program & measure how long it takes to run

- **Grading: correct solution and good performance**

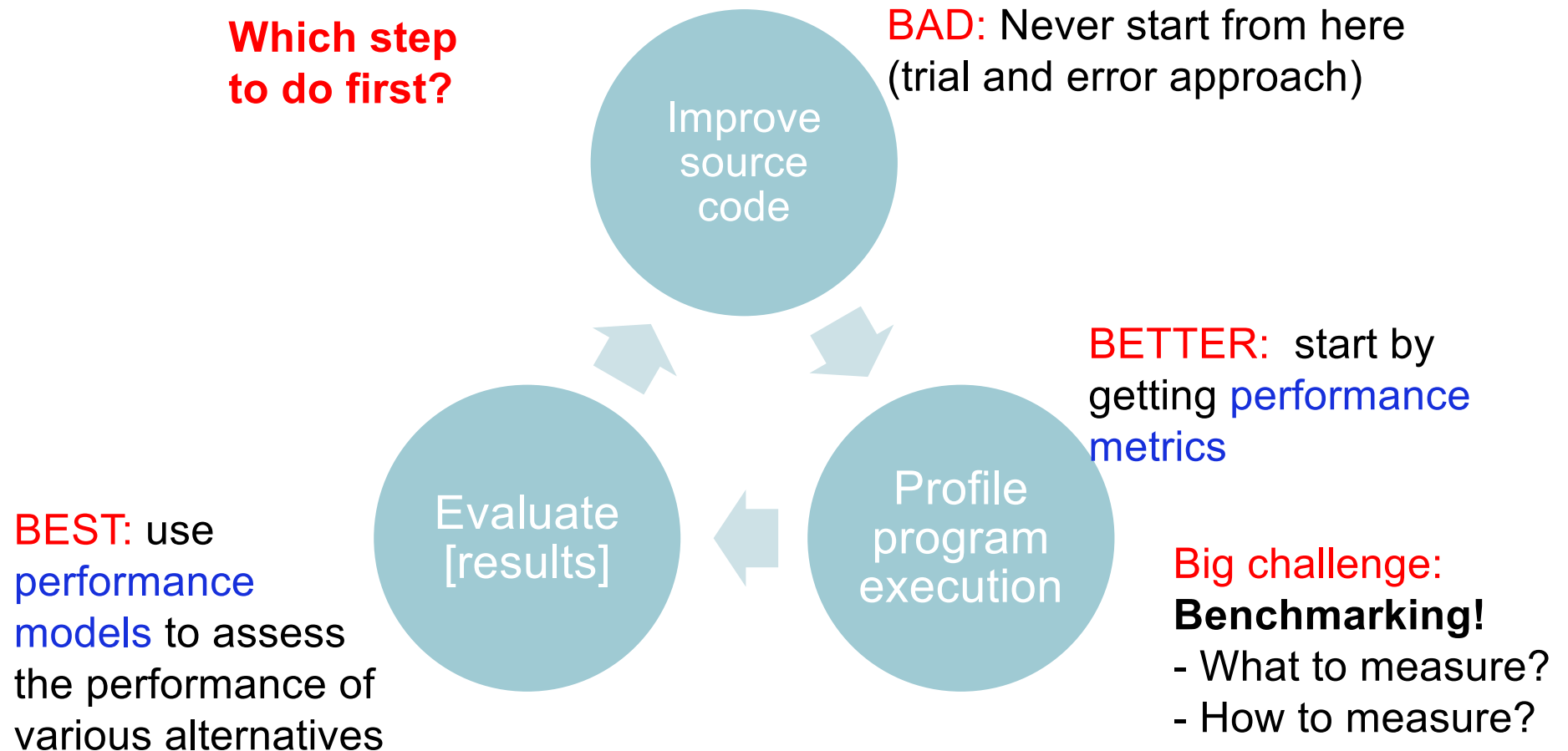
- **Emphasis on understanding performance issues**

- **experimental** set-up to assess performance
- learn to **predict** & **explain** performance results
- understand performance **trade-offs**

Performance optimization

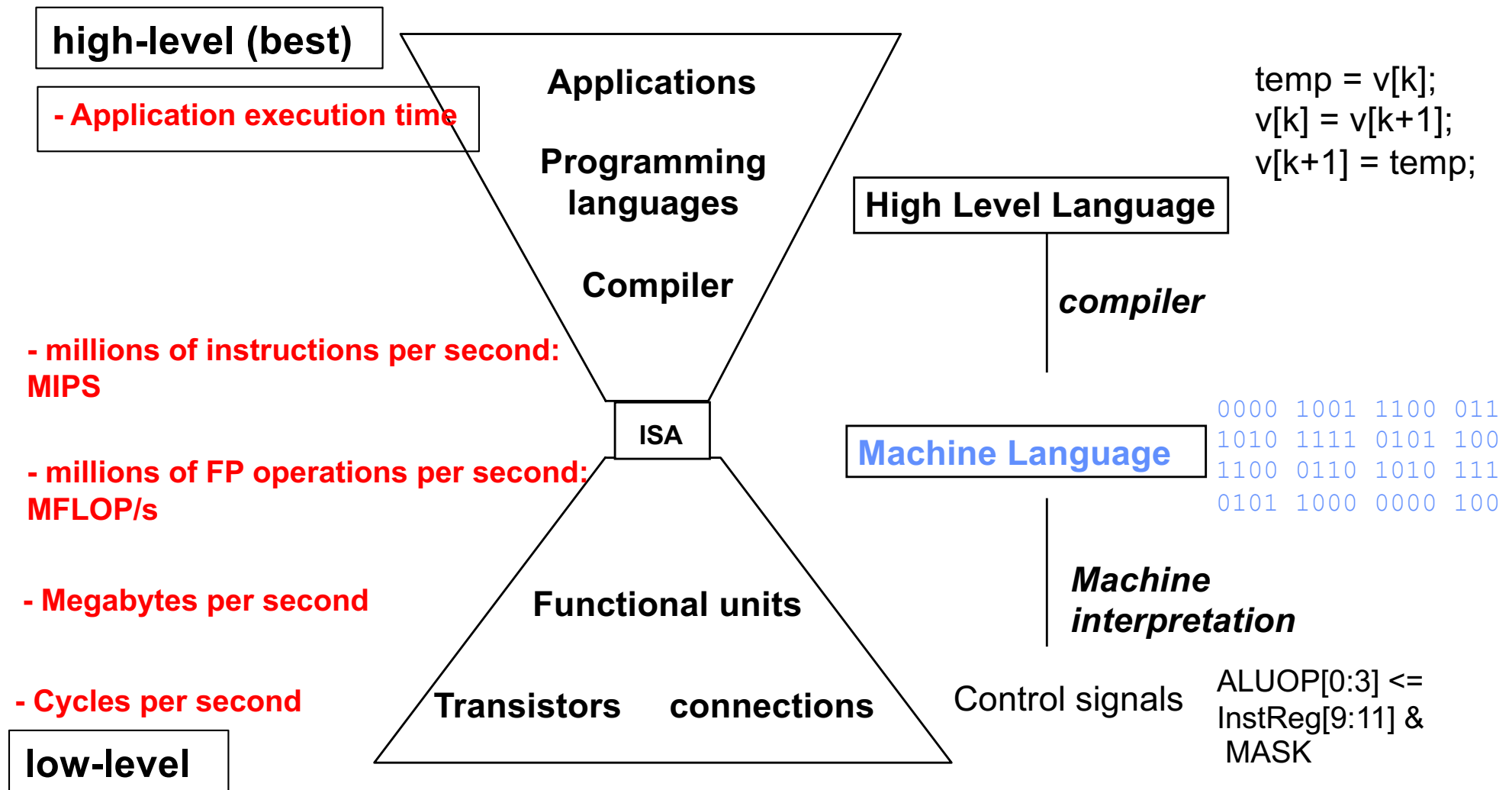
■ Typical optimization cycle of a program

Note: optimization also includes exploitation of parallelism



Performance metrics

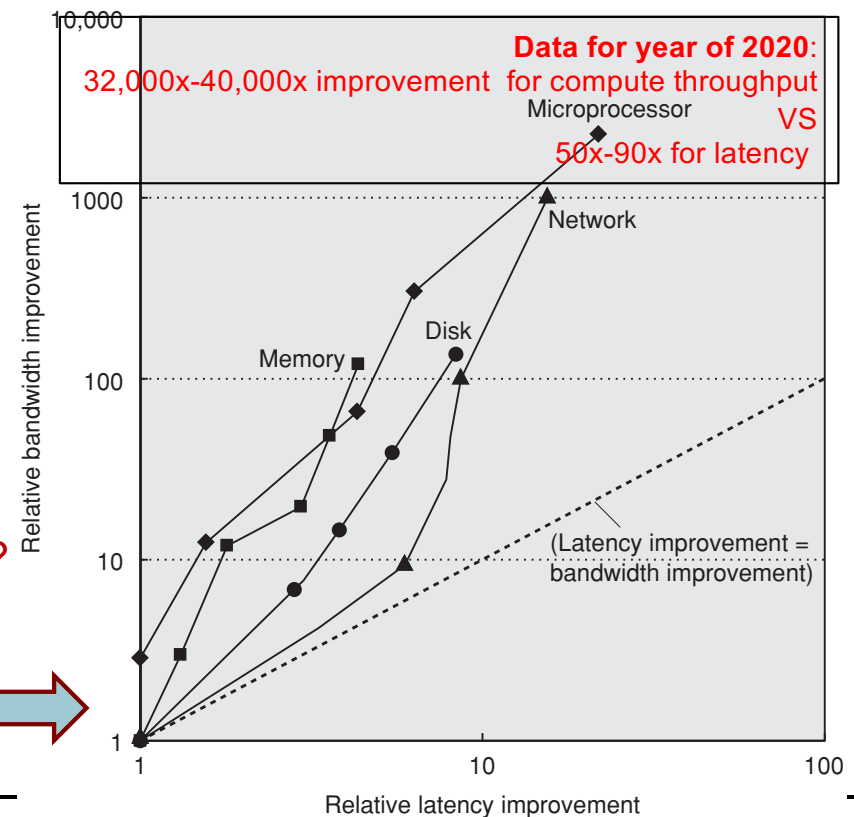
■ Can be taken at different abstraction levels



Typical performance metrics

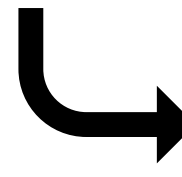
- **Latency** (or response time)
 - time required to perform an operation (e.g. program execution time)
 - **lower** value = > better performance
- **Throughput** (or Bandwidth)
 - amount of work performed per unit of time (e.g., number of FP operations per sec.)
 - **higher** value = > better performance
- In some cases:
 - $\text{Throughput} = 1/\text{Latency}$
- Examples of bandwidth:
 - **Network** (Fiber Internet): 500 Mbit/s
 - **Memory** (Apple M3): 100 GByte/s
 - **GPU** (NVIDIA) RTX 5060): 20 TFLOP/s
- **Latency vs bandwidth**
 - Fiber Internet 2 Gbit/s has 4x less latency?

Technological evolution has provided an improvement in the **throughput** of the various computer components, while **latency** has improved at a much slower rate



- The Processor Performance Equation
 - A simple model that relates high-level performance metric (CPU time) with several low-level metrics

CPU time = Instruction count × Cycles per instruction × Clock cycle time


$$CPI = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

- Example

```
movl 10, %eax
movl 0, %ecx
loop:
    addl %eax, %ecx
    decl %eax
    jnz loop
```

Estimate execution time in a computer system with a 2 GHz clock frequency and a CPI of 1.5

$$\begin{aligned} \text{CPU}_{\text{time}} &= 32 \times 1.5 \times 1/(2 \times 10^{-9}) \text{ sec} \\ &= 24 \times 10^{-9} \text{ s} = \mathbf{24 \text{ ns}} \end{aligned}$$